



NUST CHIP DESIGN CENTRE

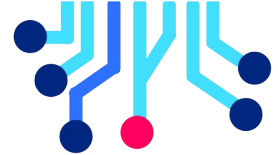
Digital Design Verification

Lab Manual # 28 – Data Wrangling

Release: 1.0

Date: 7- July -2024

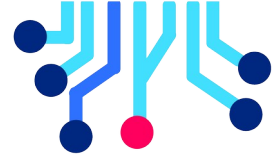
NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

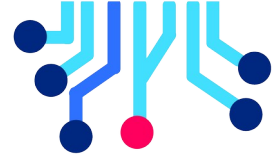
Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	07/07/2024	Muhammad Bilal	Complete manual	-



Contents

Objective.....	4
Tools and Technologies.....	4
Submission Guidelines.....	4
Overview.....	5
Task 1: Sorting an array using Bubble Sort.....	7
Task 2: Random Password Generator.....	7
References.....	7



Objective

The objective of this lab is to

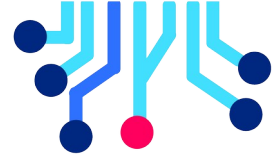
- Understand data wrangling techniques and automate the pre-processing and transformation of data using scripting.

Tools and Technologies

- Ubuntu

Submission Guidelines

You will need to submit the script files and a single report which includes the screenshots of the tested script.



Overview

Data Wrangling

Data wrangling is the process of transforming and structuring data from one raw form into a desired format with the intent of improving data quality and making it more consumable and useful for analytics.

Regular Expression:

Regular expression is a pattern for a matching string that follows some pattern:

- . means “any single character” except newline
- * zero or more of the preceding match
- + one or more of the preceding match
- [abc] any one character of a, b, and c
- (RX1|RX2) either something that matches RX1 or RX2
- ^ the start of the line
- \$ the end of the line

grep:

In the simplest terms, grep (global regular expression print) will search input files for a search string, and print the lines that match it.

If you want a wider range of regular expression commands, then you must use 'grep -E'.

Extract lines containing addi instructions:

```
cat core.txt | grep ',addi,'
```

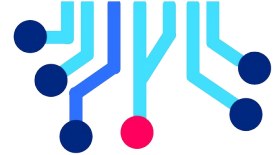
cat:

to display the contents of a file on the terminal

```
cat [OPTION] [FILE]
```

-n option to cat introduces line numbers

```
cat -n core.txt
```



Substitute:

s/old/new/

perl:

Text processing like extracting the required information from a specified text file

Use perl to extract the instr_str field for addi instructions

```
cat core.txt | grep ',addi,' | perl -pe 's/.*,(addi[^\,]*).*\1/'
```

sed = stream editor:

sed performs basic text transformations on an input stream (a file or input from a pipeline) in a

single pass through the stream, so it is very efficient.

sed to remove everything except the instr_str field for addi instructions:

```
cat core.txt | grep ',addi,' | sed -E 's/.*,(addi[^\,]*).*\1/'
```

unique and sort

Count occurrences of each unique instruction type:

```
cat core.txt | cut -d',' -f2 | sort | uniq -c
```

Show the top 10 most common instructions. The sorting is done based on first argument i.e. number of counts:

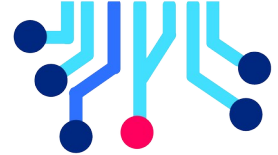
```
cat core.txt | cut -d',' -f2 | sort | uniq -c | sort -nk1,1 | tail -n10
```

awk:

An awk program operates on each line of an input file. It can have an optional BEGIN{} section of commands that are done before processing any content of the file, then the main {} section works on each line of the file, and finally there is an optional END{} section of actions that happen after the file reading has finished.

#List the top 10 most common instructions, comma-separated:

```
cat core.txt | cut -d',' -f2 | sort | uniq -c | sort -nk1,1 | tail -n10 | awk '{print $2}' | paste -sd,
```



The second field matches the pattern `/a.*i$/` in `core.txt`, you can extend your `awk` command to extract and print additional information from those lines.

```
awk -F',' ' $2 ~ /a.*i$/ { print }' core.txt
```

Task 1: Interactive Tutorial

Take this short interactive regex tutorial and attach the screenshot after completion of the tutorial in the document.

Link: <https://regexone.com/>

Task 2: Finding Words

- Write a script which prints only the assembly instruction on terminal, for any given PC value in CLI. Use **core.txt file** for this task.
- Determine how many words in **article.txt** contain at least **one 'a'** and do not end **with 'i'**. Identify the three most common last two letters of these words. You can use `sed's 'y'` command or the `tr` program to handle case insensitivity. How many **unique two-letter combinations** are there among these words? For an additional challenge, find out which two-letter combinations do not occur.

References

- University of York:
https://www-users.york.ac.uk/~mijp1/teaching/2nd_year_Comp_Lab/guides/grep_awk_sed.pdf
- MIT:
<https://missing.csail.mit.edu/2020/data-wrangling/>